



Full Stack Developer — Daniel Almanza C.C 1006122886

SECCIÓN 1 — JAVA & SPRING BOOT

30 pts

1.1 — API RESTful con Spring Boot: Arquitectura del Proyecto

El proyecto implementa Arquitectura Hexagonal (Ports & Adapters) sobre Spring Boot 3.x con Java 21. La estructura de paquetes refleja capas con responsabilidades claramente delimitadas:

Capa	Paquete / Responsabilidad
<code>domain.model</code>	Entidades de negocio puras (Product). Sin Spring. Sin JPA.
<code>domain.exception</code>	Excepciones de dominio (ProductNotFoundException).
<code>application.port.in</code>	PUERTO DE ENTRADA: ProductUseCase (interfaz con comandos tipados).
<code>application.port.out</code>	PUERTO DE SALIDA: ProductRepository (abstracción de persistencia).
<code>application.usecase</code>	ProductUseCaseImpl: orquesta lógica de negocio.
<code>infrastructure.persistence</code>	Adaptador secundario: ProductEntity (JPA), JpaProductRepository, ProductPersistenceAdapter.
<code>infrastructure.web</code>	Adaptador primario: ProductController, DTOs, GlobalExceptionHandler.
<code>infrastructure.config</code>	CacheConfig (Redis), OpenAPI config.

Endpoints Implementados

Método	Ruta	Código Exitoso	Descripción
GET	/products?page=0&size=20	200 OK	Lista paginada con Pageable
GET	/products/{id}	200 OK	Detalle por ID
POST	/products	201 Created	Crear producto + Bean Validation
PUT	/products/{id}	200 OK	Actualizar producto existente
DELETE	/products/{id}	204 No Content	Eliminar producto

1.2 — Principios SOLID Violados en Código Legacy

El código original viola 5 de 5 principios SOLID de forma directa:

#	Principio	Violación Identificada
S	SRP — Responsabilidad Única	processOrder hace TODO: consulta BD, llama API de pago, envía email y actualiza inventario. Debe dividirse en servicios especializados para evitar una sobrecarga y un mayor tiempo de respuesta.
O	OCP — Abierto/Cerrado	Para agregar un nuevo proveedor de pago hay que modificar el método directamente. Debe ser extensible sin modificación, es decir debe ser escalable para evitar tocar código.
L	LSP — Sustitución de Liskov	No se definen abstracciones, por tanto no se pueden sustituir implementaciones. No aplica directamente pero la ausencia de interfaces lo impide, ya que afectará el funcionamiento si se modifica.
I	ISP — Segregación de Interfaces	No existe ninguna interfaz. Una interfaz OrderProcessor obligaría a implementar pago, email e inventario aunque solo se necesite uno.
D	DIP — Inversión de Dependencia	OrderService depende de implementaciones concretas: DriverManager, HttpURLConnection, Session de JavaMail. No de abstracciones.

Refactorización Aplicando SOLID + Clean Architecture

```
// INTERFACES (DIP — depender de abstracciones)
public interface PaymentService {
    PaymentResult charge(Long orderId, BigDecimal amount);
}

public interface NotificationService {
    void sendOrderConfirmation(Long orderId, String email);
}

public interface InventoryService {
    void decrementStock(Long orderId);
}

public interface OrderRepository {
    Optional<Order> findById(Long orderId);
}

// CASO DE USO REFACTORIZADO (SRP + DIP + inyección por constructor)
@Service
public class OrderProcessingService {

    private final OrderRepository orderRepository;
    private final PaymentService paymentService;
    private final NotificationService notificationService;
    private final InventoryService inventoryService;

    // Inyección por constructor — testable y sin acoplamiento
    public OrderProcessingService(
        OrderRepository orderRepository,
        PaymentService paymentService,
        NotificationService notificationService,
        InventoryService inventoryService) {
        this.orderRepository = orderRepository;
    }
}
```

```

        this.paymentService      = paymentService;
        this.notificationService = notificationService;
        this.inventoryService    = inventoryService;
    }

    @Transactional
    public void processOrder(Long orderId) {
        Order order = orderRepository.findById(orderId)
            .orElseThrow(() -> new OrderNotFoundException(orderId));

        paymentService.charge(orderId, order.getTotal());
        inventoryService.decrementStock(orderId);

        // Ejecución asíncrona: no bloquear el hilo principal para el email
        CompletableFuture.runAsync(() ->
            notificationService.sendOrderConfirmation(orderId, order.getCustomerEmail())
        );
    }
}

```

SECCIÓN 2 — CLEAN ARCHITECTURE & ARQUITECTURA HEXAGONAL

25 pts

2.1 — Diseño de Arquitectura Hexagonal para Gestión de Pedidos

Conceptos Fundamentales

Núcleo / Dominio

Contiene las entidades de negocio (Order, OrderItem, OrderStatus), Value Objects, reglas de negocio invariables, y los Puertos (interfaces). No depende de Spring, JPA, ni ningún framework. Esto nunca cambia.

Puerto (Port)

Una interfaz Java que define un contrato de comunicación. Los Puertos de Entrada (Driving Ports) definen lo que el sistema puede hacer. Los Puertos de Salida (Driven Ports) definen lo que el sistema necesita del exterior, al estar ocupados o asignados, estos se darán uso en el proyecto.

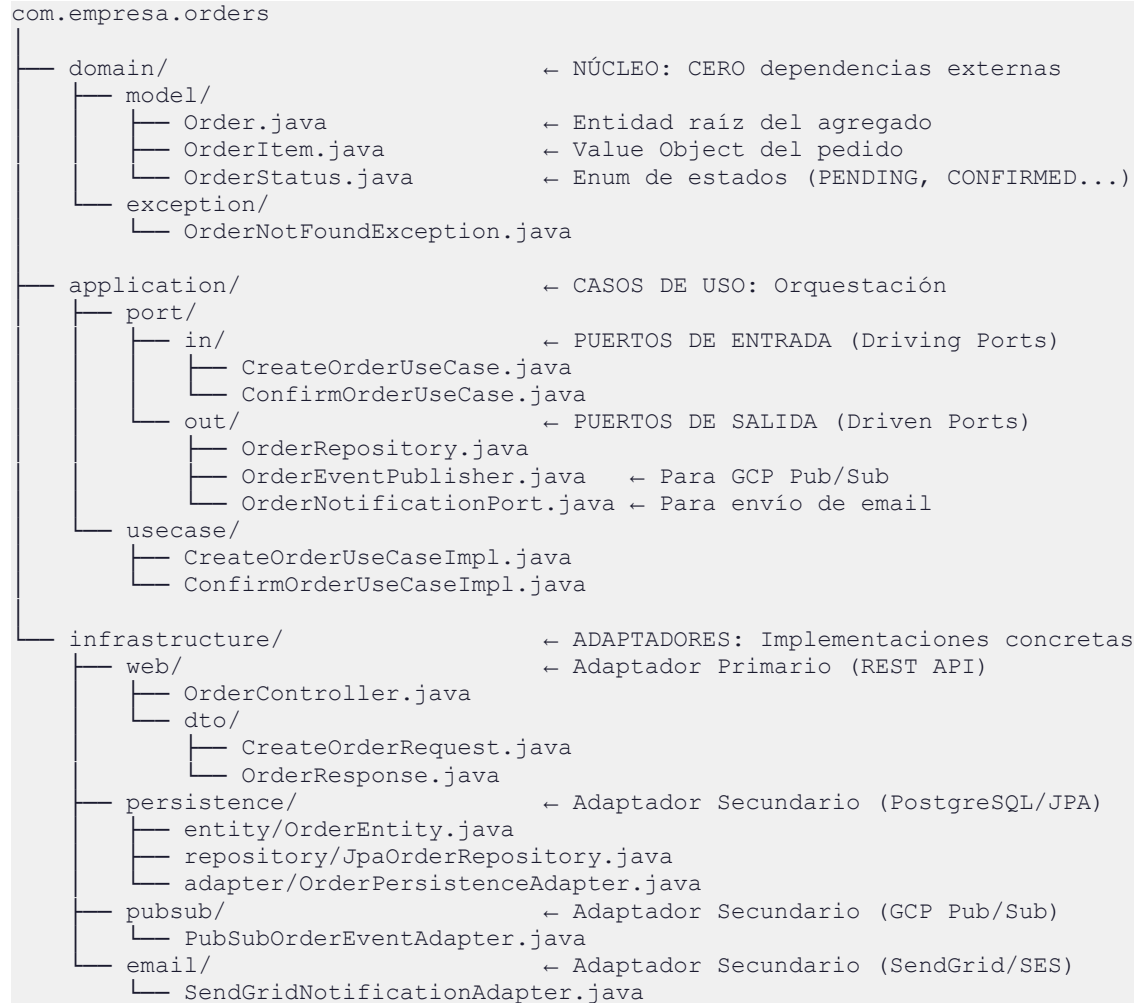
Adaptador (Adapter)

Una implementación concreta de un puerto. Un adaptador REST implementa el puerto de entrada. Un adaptador JPA implementa el puerto de repositorio. Son intercambiables sin tocar el dominio.

Casos de Uso

Viven en la capa de Aplicación. Orquestan el flujo: reciben un comando del adaptador primario, ejecutan la lógica del dominio y delegan efectos secundarios a los puertos de salida.

Estructura de Paquetes — com.empresa.orders



¿Por qué GCP Pub/Sub es un Adaptador Secundario?

Justificación Técnica

GCP Pub/Sub es un sistema externo al que nuestro dominio le envía mensajes (driven/secondary). El dominio define un puerto de salida `OrderEventPublisher` con el método `publish(OrderConfirmedEvent)`. El adaptador `PubSubOrderEventAdapter` implementa ese puerto usando el SDK de Google Cloud. Si mañana migramos a AWS SNS o RabbitMQ, solo cambiamos el adaptador. El dominio y los casos de uso permanecen intactos. Esta es la esencia del principio de inversión de dependencias aplicado a nivel arquitectónico.

3.1 — Control de Concurrency e Integridad en Stock

Bloqueo Optimista (@Version)	Bloqueo Pesimista (SELECT FOR UPDATE)
Asume que las colisiones son raras. No bloquea la BD. Al guardar, verifica que la versión no haya cambiado. Si cambió → <code>OptimisticLockException</code> . Ideal para lecturas frecuentes y escrituras irregulares.	Bloquea la fila en BD inmediatamente con <code>SELECT FOR UPDATE</code> . Otros hilos se bloquean hasta que la transacción libera el lock. Ideal para alta concurrencia transaccional con conflictos frecuentes.

Elección para Cyber-Day: Bloqueo Pesimista + Optimista Combinados

Para un evento de alta demanda donde múltiples usuarios compiten por el MISMO producto (stock=1), el bloqueo pesimista es la elección primaria porque garantiza que solo UNA transacción puede leer-modificar-guardar el stock a la vez. Sin embargo, para operaciones de lectura masiva (GET /products) se mantiene el bloqueo optimista con @Version para detectar actualizaciones concurrentes sin degradar la lectura.

Implementación con Bloqueo Pesimista — Sección Crítica Protegida

```
// JpaProductRepository.java
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM ProductEntity p WHERE p.id = :id")
Optional<ProductEntity> findByIdWithPessimisticLock(@Param("id") Long id);

// ProductUseCaseImpl.java — Compra con protección de concurrencia
@Transactional(isolation = Isolation.READ_COMMITTED)
public void purchaseProduct(Long productId, int quantity) {
    // SECCIÓN CRÍTICA: El SELECT FOR UPDATE bloquea la fila en PostgreSQL.
    // Otros hilos esperan en cola hasta que esta transacción finalice.
    ProductEntity product = jpaRepository
        .findByIdWithPessimisticLock(productId)
        .orElseThrow(() -> new ProductNotFoundException(productId));

    if (product.getStock() < quantity) {
        // HTTP 422 Unprocessable Entity: regla de negocio violada
        throw new InsufficientStockException(productId, product.getStock(), quantity);
    }

    product.setStock(product.getStock() - quantity);
    jpaRepository.save(product);
    // Al finalizar @Transactional → COMMIT → se libera el lock
}

// HTTP Responses por tipo de error de concurrencia:
// OptimisticLockException → 409 Conflict (reintente)
// InsufficientStockException → 422 Unprocessable Entity
// Lock timeout → 503 Service Unavailable (con Retry-After header)
```

Estrategia de resiliencia: En el cliente, implementar reintentos exponenciales con jitter (max 3 intentos, delay inicial 100ms) para colisiones 409. Para stock agotado 422, retornar inmediatamente sin reintentar, se evita la máxima cantidad de request para evitar el spam y baja del servidor.

4.1 — SQL Avanzado Multi-Motor

a) Top 5 Clientes por Valor Acumulado (últimos 30 días)

```
-- =====
-- a) TOP 5 CLIENTES — Mayor valor en compras (últimos 30 días)
-- Compatible: PostgreSQL y OracleSQL
-- =====

-- POSTGRESQL
WITH customer_stats AS (
    SELECT
        c.id,
        c.name,
        c.email,
        COUNT(DISTINCT o.id)          AS total_orders,
        SUM(o.total)                  AS total_spent,
        ROUND(AVG(o.total)::NUMERIC, 2) AS avg_ticket
    FROM customers c
    JOIN orders o ON o.customer_id = c.id
    WHERE o.status != 'CANCELLED'
        AND o.created_at >= NOW() - INTERVAL '30 days'
    GROUP BY c.id, c.name, c.email
)
SELECT
    RANK() OVER (ORDER BY total_spent DESC) AS ranking,
    id, name, email, total_orders, total_spent, avg_ticket
FROM customer_stats
ORDER BY total_spent DESC
FETCH FIRST 5 ROWS ONLY;

-- ORACLE SQL (diferencias: SYSDATE, TRUNC, TO_CHAR)
-- WHERE o.created_at >= SYSDATE - 30
-- ROUND(AVG(o.total), 2) (sin cast)
```

b) Productos con Stock Crítico y Ventas > 50 unidades

```
-- =====
-- b) PRODUCTOS CON STOCK CRÍTICO + ventas > 50 uds (último mes)
-- =====

-- POSTGRESQL
WITH monthly_sales AS (
    SELECT
        oi.product_id,
        SUM(oi.quantity) AS units_sold
    FROM order_items oi
    JOIN orders o ON o.id = oi.order_id
    WHERE o.status NOT IN ('CANCELLED', 'RETURNED')
        AND o.created_at >= DATE_TRUNC('month', NOW())
        AND o.created_at < DATE_TRUNC('month', NOW()) + INTERVAL '1 month'
    GROUP BY oi.product_id
    HAVING SUM(oi.quantity) > 50
)
SELECT
    p.id,
```

```

    p.name,
    p.category,
    p.stock,
    ms.units_sold
FROM products p
JOIN monthly_sales ms ON ms.product_id = p.id
WHERE p.stock < 10
ORDER BY p.stock ASC, ms.units_sold DESC;

```

c) Reporte de Ventas Diarias por Categoría — Días en Cero Incluidos

```

-- =====
-- c) VENTAS DIARIAS (últimos 7 días) por categoría — días en CERO incluidos
-- POSTGRESQL (usa GENERATE_SERIES)
-- =====

WITH date_series AS (
    SELECT generate_series(
        CURRENT_DATE - INTERVAL '6 days',
        CURRENT_DATE,
        '1 day'::INTERVAL
    )::DATE AS sale_date
),
categories AS (
    SELECT DISTINCT category FROM products
),
date_category_grid AS (
    -- Producto cartesiano: todas las combinaciones fecha x categoría
    SELECT ds.sale_date, cat.category
    FROM date_series ds
    CROSS JOIN categories cat
),
daily_sales AS (
    SELECT
        o.created_at::DATE AS sale_date,
        p.category,
        SUM(oi.quantity * oi.unit_price) AS revenue,
        SUM(oi.quantity) AS units_sold,
        COUNT(DISTINCT o.id) AS total_orders
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.id
    JOIN products p ON p.id = oi.product_id
    WHERE o.status NOT IN ('CANCELLED', 'RETURNED')
    AND o.created_at >= CURRENT_DATE - INTERVAL '6 days'
    GROUP BY o.created_at::DATE, p.category
)
SELECT
    dcg.sale_date,
    dcg.category,
    COALESCE(ds.revenue, 0) AS revenue,
    COALESCE(ds.units_sold, 0) AS units_sold,
    COALESCE(ds.total_orders, 0) AS total_orders
FROM date_category_grid dcg
LEFT JOIN daily_sales ds
    ON ds.sale_date = dcg.sale_date
    AND ds.category = dcg.category
ORDER BY dcg.sale_date DESC, dcg.category;

-- ORACLE: reemplazar GENERATE_SERIES con:
-- SELECT TRUNC(SYSDATE) - (LEVEL-1) AS sale_date
-- FROM DUAL CONNECT BY LEVEL <= 7

```

4.2 — Redis como Capa de Caché Empresarial

Estrategia de Caché — Decisiones de Diseño

TTL de 1 hora para productos individuales (cache "products"). TTL de 30 minutos para listas paginadas (cache "product-lists", más volátiles). Al crear/actualizar/eliminar un producto, se invalidan AMBAS cachés con @CacheEvict. Mitigación de Cache Stampede: TTL con jitter aleatorio (± 5 minutos), lock distribuido con Redisson para que solo 1 hilo recargue la caché al expirar, y Probabilistic Early Expiration (PER) para distribución de renovación.

```
// CacheConfig.java — TTL diferenciado por caché
@Configuration @EnableCaching
public class CacheConfig {
    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration defaults = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofHours(1))
            .disableCachingNullValues()
            .serializeValuesWith(/* Jackson JSON serializer */);

        return RedisCacheManager.builder(factory)
            .cacheDefaults(defaults)
            .withInitialCacheConfigurations(Map.of(
                "products", defaults.entryTtl(Duration.ofHours(1)),
                "product-lists", defaults.entryTtl(Duration.ofMinutes(30))
            ))
            .build();
    }
}

// CachedProductService.java — Uso declarativo
@Cacheable(value = "products", key = "#id")
public Product getProductById(Long id) { ... } // Hit O(1), miss → BD

@Caching(evict = {
    @CacheEvict(value = "products", key = "#id"),
    @CacheEvict(value = "product-lists", allEntries = true)
})
public Product updateProduct(Long id, ...) { ... } // Invalida al escribir
```

SECCIÓN 5 — GOOGLE CLOUD PLATFORM & DOCKER

10 pts

5.1 — Arquitectura de Despliegue Cloud Native

Elección de Servicio de Cómputo en GCP

Criterio	Cloud Run	GKE	App Engine
Costo	Pago por request	Alto (nodos 24/7)	Medio
Escala Automática	0 a N instancias	Manual + HPA	Automático
Overhead Operacional	Mínimo	Alto (k8s)	Bajo
Recomendado para	Microservicios stateless	Orquestación compleja	Apps monolíticas

Elección: Cloud Run

Para este e-commerce de microservicios, Cloud Run es la opción óptima. Es serverless, escala automáticamente a 0 cuando no hay tráfico (costo mínimo) y a cientos de instancias en picos (Cyber-Day), no requiere gestión de Kubernetes, y tiene integración nativa con GCP Secret Manager, Cloud SQL y Artifact Registry, en ese caso para apps monolíticas en donde todo esta unificado es lo más óptimo

Dockerfile Multi-Stage Optimizado para Producción

```
# ETAPA 1: Build — JDK completo para compilar
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /build
COPY pom.xml .
RUN mvn dependency:go-offline -q          # Cachea dependencias
COPY src ./src
RUN mvn clean package -DskipTests -q

# ETAPA 2: Extract Layered JAR (optimiza caché de Docker)
FROM eclipse-temurin:21-jdk-alpine AS extractor
WORKDIR /app
COPY --from=builder /build/target/*.jar app.jar
RUN java -Djarmode=layertools -jar app.jar extract

# ETAPA 3: Runtime final — solo JRE, imagen mínima
FROM eclipse-temurin:21-jre-alpine AS runtime
WORKDIR /app

# Capas ordenadas de menor a mayor frecuencia de cambio
COPY --from=extractor /app/dependencies/ ./
COPY --from=extractor /app/spring-boot-loader/ ./
COPY --from=extractor /app/snapshot-dependencies/ ./
COPY --from=extractor /app/application/ ./

# SEGURIDAD: usuario no-root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
RUN chown -R appuser:appgroup /app
USER appuser

EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0",
  "org.springframework.boot.loader.launch.JarLauncher"]
```

Gestión de Secretos — GCP Secret Manager

```
# En Cloud Run, los secretos se montan como variables de entorno
# NO como archivos en disco ni hardcodeados en el código/Dockerfile

gcloud run deploy ecommerce-api \
  --set-secrets="DB_PASSWORD=db-password:latest" \
  --set-secrets="REDIS_URL=redis-url:latest" \
  --set-secrets="JWT_SECRET=jwt-secret:latest"

# En Spring Boot, se consume como variable de entorno estándar:
# spring.datasource.password=${DB_PASSWORD}
# Alternativa: spring-cloud-gcp-starter-secretmanager para acceso directo
```

Pipeline CI/CD — GitHub Actions + Cloud Run

```
# .github/workflows/deploy.yml
name: CI/CD Pipeline - NYXN E-commerce

on:
  push:
    branches: [main]

env:
  PROJECT_ID: ${ secrets.GCP_PROJECT_ID }
  REGION: us-central1
  IMAGE: gcr.io/${ secrets.GCP_PROJECT_ID }/ecommerce-api

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: 21, distribution: temurin }
      - run: mvn test -q

  build-and-deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with:
          credentials_json: ${ secrets.GCP_SA_KEY }
      - run: gcloud auth configure-docker
      - name: Build & Push Docker Image
        run: |
          docker build -t $IMAGE:${ github.sha } .
          docker push $IMAGE:${ github.sha }
      - name: Deploy to Cloud Run
        run: |
          gcloud run deploy ecommerce-api \
            --image $IMAGE:${ github.sha } \
            --region $REGION \
            --platform managed \
            --allow-unauthenticated \
            --set-secrets="DB_PASSWORD=db-password:latest" \
            --min-instances=1 \
            --max-instances=100
```

SECCIÓN 6 — NODE.JS, NEXT.JS & INTELIGENCIA ARTIFICIAL

10 pts

6.1 — Microservicio de Notificaciones con Patrón Strategy

El microservicio en TypeScript implementa el patrón Strategy de forma rigurosa: la interfaz NotificationStrategy define el contrato, cada canal (Email, SMS, Push) es una estrategia concreta intercambiable, y el NotificationContext selecciona la estrategia en runtime basado en el campo channel del payload. **Ver código completo en la ruta:** node-notification-service/src/notification.service.ts

Ventajas del Patrón Strategy en este Contexto

- Open/Closed: Para agregar WhatsApp, solo se crea WhatsAppStrategy y se registra. CERO cambios en el endpoint /notify.
- Testabilidad: Cada estrategia se prueba de forma aislada con mocks del proveedor externo.
- Separación de responsabilidades: El routing y la ejecución están completamente separados con el fin de evitar lentitud en los procesos.
- Runtime flexibility: La estrategia se selecciona dinámicamente, no en compilación.

6.2 — Agente de IA con Claude (Anthropic API)

Arquitectura del Agente — Tool Use vs RAG

La elección es Tool Use (Function Calling), NO RAG, para el flujo transaccional de órdenes. RAG es óptimo para búsqueda semántica en bases de conocimiento. Para acciones transaccionales como consultar estado de pedido, aplicar descuentos, o generar resúmenes de comportamiento de cliente, Tool Use permite que Claude ejecute acciones concretas de forma estructurada y leer procesos del momento.

Tareas Delegadas al LLM

- Análisis de sentimiento de reseñas y clasificación automática de reclamaciones por urgencia.
- Generación de resúmenes ejecutivos de comportamiento de compra para el equipo de ventas.
- Respuestas automáticas a consultas de estado de pedido (con acceso al historial vía Tool Use).
- Detección de patrones de fraude en órdenes sospechosas usando análisis contextual.
- Generación de descripciones optimizadas para SEO de nuevos productos.

Implementación Segura — Tool Use (Function Calling)

```
// Backend: definición de herramientas disponibles para Claude
const tools = [
  {
    name: "get_order_status",
    description: "Consulta el estado actual de un pedido por su ID",
    input_schema: {
      type: "object",
      properties: {
        order_id: { type: "string", description: "ID del pedido" }
      },
      required: ["order_id"]
    }
  },
  {
    name: "apply_discount",
    description: "Aplica un descuento a un pedido pendiente",
    input_schema: {
      type: "object",
      properties: {
        order_id: { type: "string" },
        discount_percentage: { type: "number", minimum: 0, maximum: 50 }
      },
      required: ["order_id", "discount_percentage"]
    }
  }
]
```

```

];

// SEGURIDAD BIDIRECCIONAL:
// 1. API Key de Anthropic → solo en variables de entorno del backend (NUNCA frontend)
// 2. Validación de outputs: el backend valida que tool_use sea un tool registrado
// 3. Autorización: antes de ejecutar apply_discount, verificar permisos del usuario
// 4. Auditoría: cada invocación de tool_use se loguea con userId + timestamp
// 5. Rate limiting: máximo N llamadas al LLM por usuario por hora

async function processWithClaude(userMessage, userId) {
  const response = await anthropic.messages.create({
    model: "claude-sonnet-4-20250514",
    max_tokens: 1024,
    tools,
    messages: [{ role: "user", content: userMessage }]
  });

  if (response.stop_reason === "tool_use") {
    const toolUse = response.content.find(b => b.type === "tool_use");

    // Ejecutar en backend seguro, no exponer a cliente
    const result = await executeToolSecurely(toolUse.name, toolUse.input, userId);
    // ... enviar resultado de vuelta a Claude para respuesta final
  }
}

```

¿Por qué Tool Use y no RAG?

RAG (Retrieval Augmented Generation) es ideal para preguntas sobre documentación, catálogos de productos, o bases de conocimiento estático. En el flujo de gestión de órdenes necesitamos ACCIONES en tiempo real: consultar el estado de una orden específica, aplicar descuentos, actualizar estados. Tool Use permite que Claude razone sobre qué herramienta usar, con qué parámetros, y luego el backend ejecuta la acción con plena validación de seguridad. El LLM nunca accede directamente a la BD — solo invoca herramientas del backend que ejecutan la lógica de negocio con las validaciones de autorización correspondientes para el buen uso y mayor practicidad